

Agentic Coding

Sven Helmer

Kathrin Paschen

Chapter 1

Introduction

Motivation

- There's a lot of hype around vibe coding at the moment

"No more humans coding in 5 years."

Emad Mostaque

"AI will write most of the code soon and perform better than top coders."

Mark Zuckerberg

"It's our job to create computing technology such that nobody has to program."

Jensen Huang

Motivation (2)

- This is not the first hype we have seen:

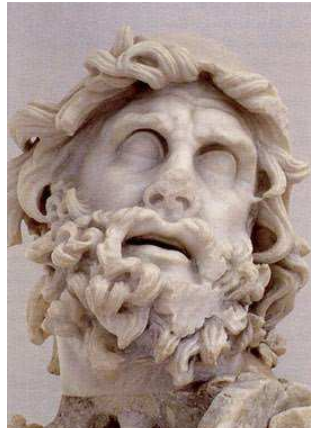


Things are Definitely Changing

- In the foreseeable future, AI will not replace developers
 - A developer using AI will replace one who doesn't
 - and developers (already) work differently
- Currently, AI boosts the skills of experts
- It will not save you if you have no clue what you're doing

Basic Terminology

- Coding with AI covers a whole spectrum
 - Vibe coding: prompt and hope for the best
 - Agentic coding: a systematic approach that still utilizes software development principles



vibe coding

agentic coding

Vibe Coding

- Vibe coding typically looks like the following:
 - Write a prompt, e.g. “Build an app that does X.”
 - LLM generates some code
 - User tries it out, notices some things that do not work
 - User writes another prompt requesting changes
 - LLM changes code
 - Wash, rinse, repeat

Agentic Coding

- Gives an LLM specific software development tasks, such as:
 - Specify goal
 - Devise a plan: break up overall task into subtasks
 - Generate code
 - Validate code (for example by running tests)
 - Revise plan or implementation
 - Repeat until satisfied

Vibe-Coding vs. Agentic Coding

- Vibe coding isn't always bad, and agentic coding isn't always the right approach
- Vibe coding:
 - makes sense for rapid prototyping
 - is for quickly trying out things
- Agentic coding
 - is a thorough process
 - creates reliable code

What Can Go Wrong?

- AI gets stuck in infinite loops
- Delivers incorrect solutions confidently
- Cheats: optimize for validation such as benchmarks
- Implements what you said, not what you meant
 - General problem, existed before AI

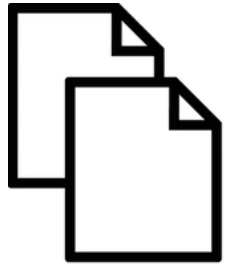
Summary

- Then: translate ideas into code (programming)
- Now: translate ideas into prompts (vibe-coding)
- Then: (systematically) applying principles and methods to create reliable software (software engineering)
- Now: give an AI (software-engineering) goals and constraints to create software (agentic coding)
- Important question: how much of the software-engineering process can be delegated?

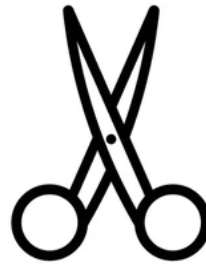
Chapter 2

The Environment

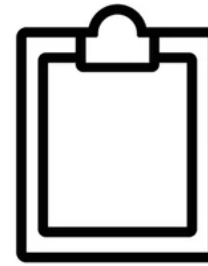
Basics



Copy



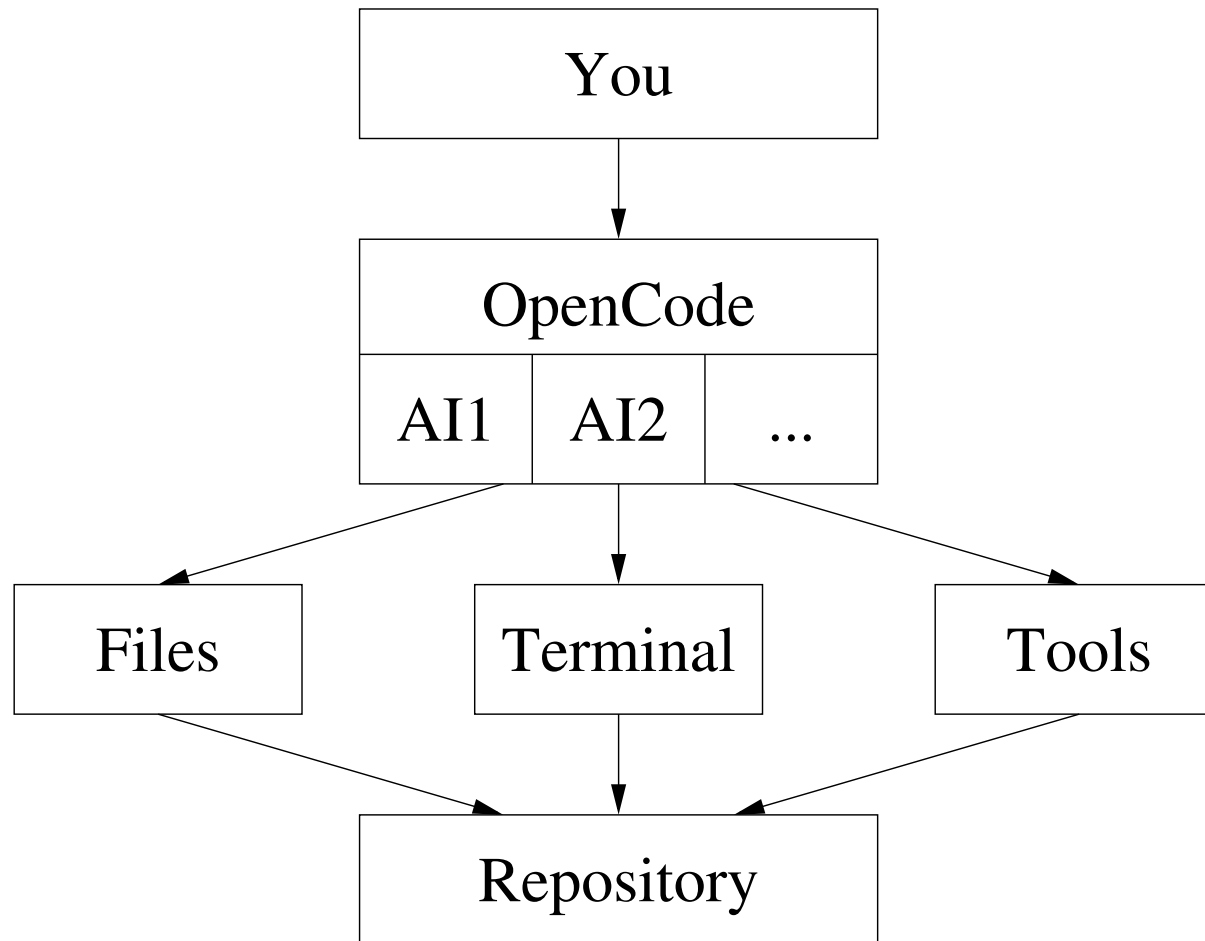
Cut



Paste

- What we don't want to do:
 - Prompting LLM on a web interface
 - Then copy/pasting code into file
- We use a more sophisticated environment

Overview



OpenCode

- OpenCode is an open-source coding tool
- Connects LLMs to development tools
- Model-agnostic: can interact with (almost) every model you want
- By default, code is created and stored locally

The logo for OpenCode, featuring the word "open" in a light gray, lowercase, sans-serif font, followed by "code" in a bold, black, lowercase, sans-serif font. The letters are slightly shadowed, giving a 3D effect.

Interaction with Files

- OpenCode is not an editor:
 - You can use any editor you want (VS Code, Emacs, IntelliJ) to look at the files
 - OpenCode executes commands in a terminal
- With these commands it can manipulate files and use other tools
 - This has direct implications for security

Repositories

- A repository is where a version control system (VCS) stores its data
- A VCS tracks and manages changes to files
 - Heavily used in managing source code
- Allows to revert to a previous state
- Facilitates collaboration

Model Context Protocol (MCP)

- We just mention it for sake of completeness
- It is an open standard specifying how LLMs share data with external tools
- MCP offers a standardized interface for
 - reading files
 - executing functions
 - handling prompts

Summary

- This chapter is kept short on purpose
- We show how to set up and use this environment in hands-on exercises



THEORY TELLS. PRACTICE TEACHES.

Chapter 3

A Mental Model of LLMs and Agentic Coding

Introduction

- The aim is not to give a detailed technical description of LLMs (and the agents utilizing them)
 - You don't need to build an LLM or agent,
 - but to understand how they react to your input
- In a nutshell, an LLM is autocomplete on steroids
 - It doesn't look up an answer
 - It generates its response incrementally, depending on the context and the data it was trained on

Autocomplete vs. LLM

- If you type a message on your phone, it tries to predict how the message continues
 - Uses a dictionary and probabilities for continuations
- An LLM works at a different scale
 - Instead of a dictionary, uses a neural network with billions or trillions of parameters
 - Not just larger, also tries to generalize
 - Utilizes a much larger context (thousands of words)
 - Depending on the setting, it may not always return the most probable answer

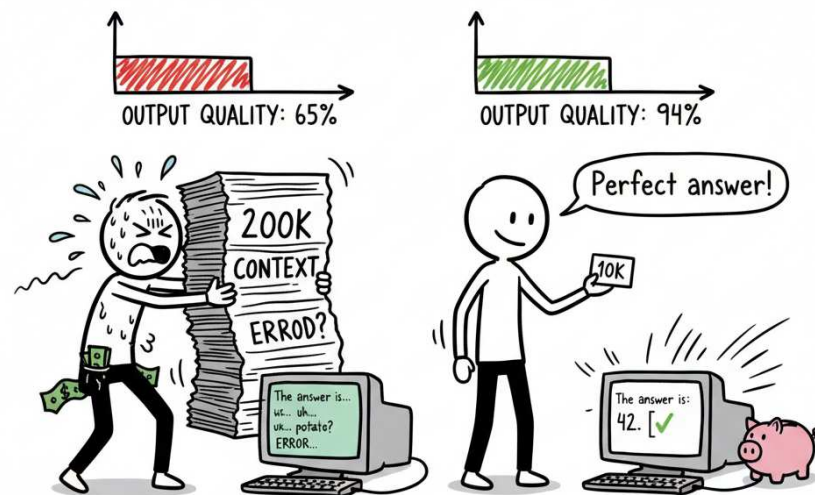
Tokens

- An LLM doesn't work with words, uses tokens instead
- We only give a brief introduction to tokens
- What do tokens look like?
 - (Short) words, e.g. *house*
 - Parts of words, e.g. *un con stitu tion al*
 - Punctuation, whitespace, emojis
- Tokens are used
 - to avoid vocabulary explosion
 - identify new words
 - describe concepts independent of language

Generating an Answer

- Context: current prompt and previous input/output
- An LLM then generates a plausible continuation:
context → next token → next token → next token
- A generated token is added to the context
- The *context window* plays a crucial role

THE CONTEXT WINDOW PARADOX



Context Window

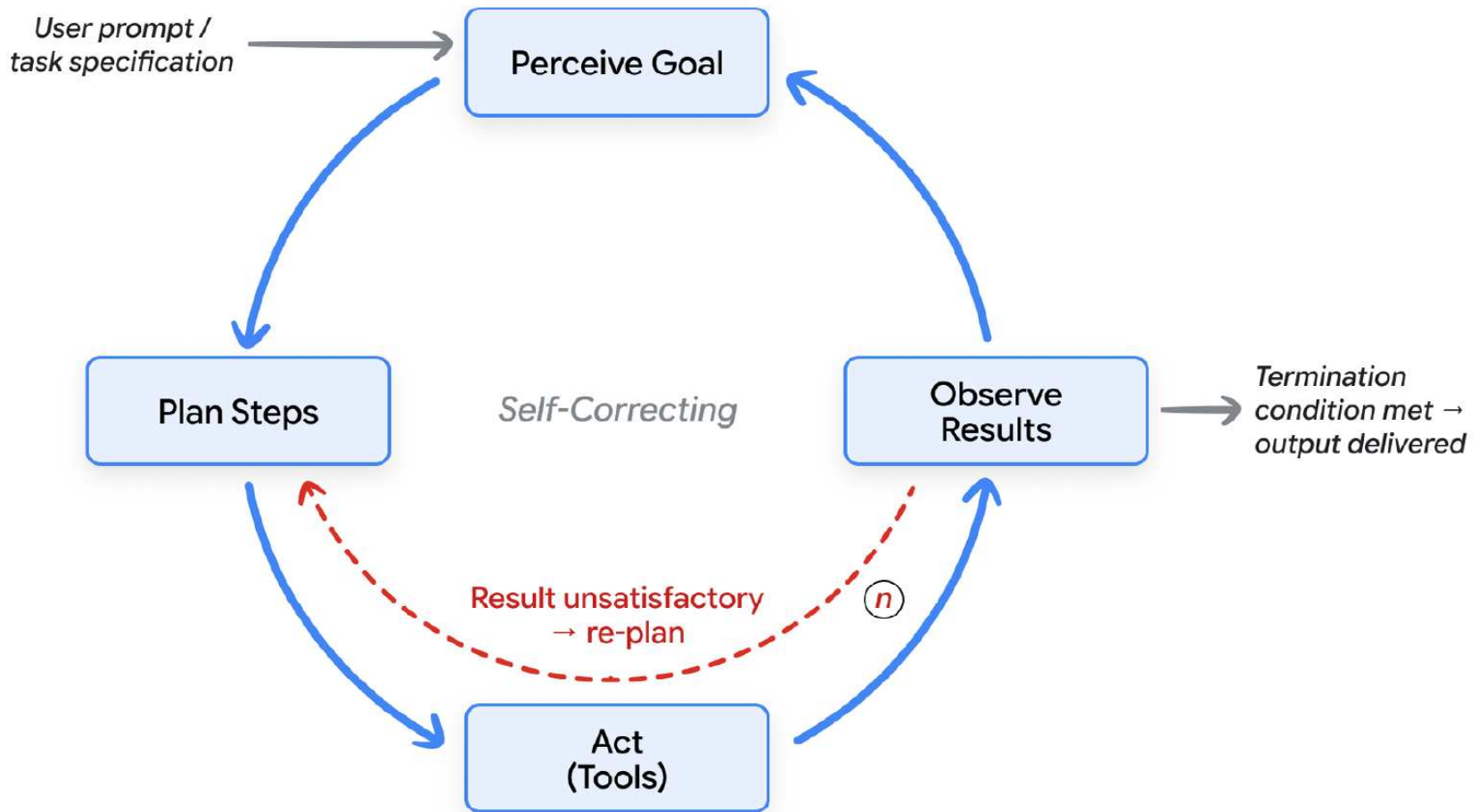
- Process is driven by the content of the context window
 - This window is not infinitely large
 - Higher recall at edges (beginning/end)
- Some similarity to our brain's working memory
 - Can hold seven chunks of information (+/-2)
 - Serial position effect (easier to remember start/end)
- Takeaway: if we want good results, we have to provide the right context

From Chatbot to Agent

- We don't want to use the LLM in the following way:
prompt → answer
- First of all, there's an issue with reliability
 - LLMs can be confidently incorrect
- Given a prompt (and all their learned patterns), an LLM gives plausible answers
 - LLMs do not know what the outside world looks like
- We want evidence for correctness
 - Idea: give LLMs access to tools for checking

From Chatbot to Agent (2)

- We want to follow a loop:



From Chatbot to Agent (3)

- In the context of coding, we provide tools such as
 - Requirement/specification documents
 - Code files
 - Tests
 - Current repository and history
 - External documentation
- In agentic coding, the LLM itself is not the agent
 - The agent is the system driving the above loop and using the LLM

Summary

- We covered a basic mental model of LLMs and agents
- Helps you in answering questions like:
 - Why did the LLM/agent just do this?
 - How can I make it behave differently?
- We briefly explained how LLMs/agents interact with the outside world

Chapter 4

From Problem to Specification

Introduction



- LLMs/agents don't magically know what you mean
- An agent can only implement
 - what you have specified
 - or what it can safely infer
- If your specification has gaps, the agent will fill them
- Results can range from *hilarious* to *dangerous*

A Cautionary Tale

- Assume you want to analyze some experimental data
- You enter the following as a prompt:
“Write a program that analyzes experimental data.”
- What do you expect the agent to produce?

THAT'S THE UGLIEST MESS
OF CODE I'VE EVER SEEN.
WHAT ON EARTH ARE YOU
WORKING ON?



A Cautionary Tale (2)

- To generate sensible code, an agent needs to know:
 - What kind of experiment?
 - What do we mean by “analyze”?
 - Which questions do we want answered?
 - What does the data look like?
 - What output do we want?
 - Who will use the result?
 - How do we check correctness?
- We need to define a lot of different aspects

Covering Different Aspects

- **Problem:** what is the general goal?
- **Requirements:** what should the program do?
- **Specification:** how should it do it?
- **Constraints:** what must/mustn't it do?
- **Acceptance criteria:** how do we check it's correct?
- **Examples:** what do concrete use cases look like?

Back to Our Example

- Problem
 - Analyze my experimental data
- Requirements
 - Load CSV files
 - Calculate summary statistics
 - Identify outliers
 - Produce result file and plots

Back to Our Example (2)

● Specification

- For each CSV file
 - group measurements by experimental condition
 - compute mean, median and standard deviation
 - identify observations more than 3 standard deviations from the group mean
 - ...

● Constraints

- For the plots, use matplotlib
- Don't use Python versions older than 3.14
- Don't modify the original data files

Back to Our Example (3)

- Acceptance criteria
 - A handcrafted example (with known output) produces the same output
 - `results.csv` has to pass some statistical tests
- Examples
 - Provide typical snippets of experimental data
 - Also include some edge cases

Resolving Ambiguities

- What happens if we don't provide this information?
- LLMs always give an answer
- They resolve ambiguities by filling in the gaps
 - Based on the most plausible options from training
- This does not usually get you what you want.



Decomposition

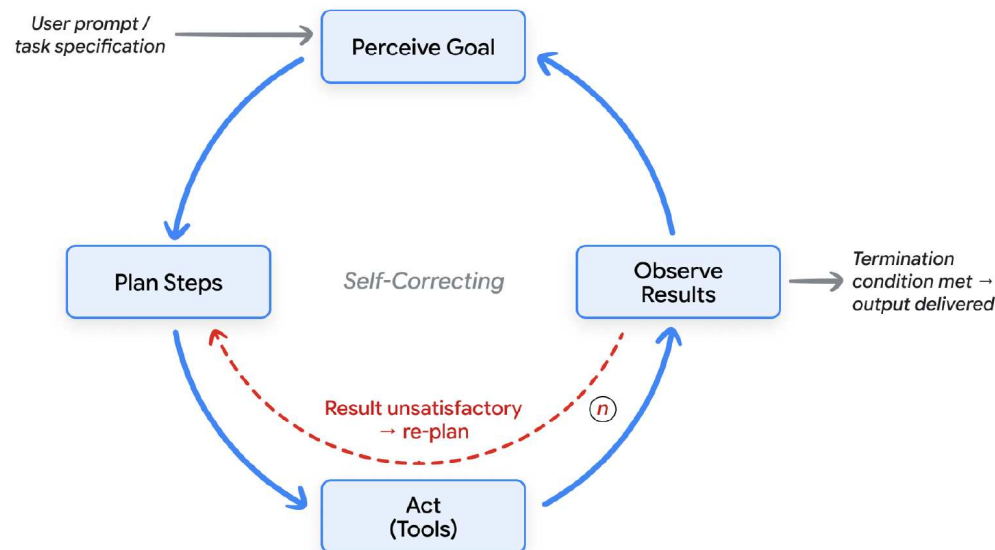
- Once the specification is done, ask the following:
Can this be broken down into smaller subtasks?
- For analyzing experimental data, this could look like
 - Load data (specifying input format + validation)
 - Group observations
 - Calculate statistics
 - Detect outliers
 - Write results
 - Generate plots
 - Decide whether to accept analysis

Decomposition (2)

- Gives you checkpoints
 - Instead of producing thousands of lines of code,
 - implements and verifies one part at a time
- Also helps with resolving ambiguities
 - Can now also be done for one part at a time
- We talk more about decomposition in the next chapter

Mapping to Agentic Loop

- Maps nicely onto the loop from the previous chapter:
 - Perceive Goal – Problem
 - Plan Steps – Specification
 - Act – Implement
 - Observe Results – Check Acceptance Criteria
- If it isn't working, refine specification



Summary

- Agentic coding is not about finding the “magic” prompt
- It’s about (systematically) getting rid of ambiguities
 - If we don’t, an LLM will fill gaps on its own
- So, if things don’t work
 - Don’t ask an agent to fix the code
 - Instead, go over the specification

“On two occasions I have been asked, – ‘Pray, Mr. Babbage, if you put into the machine wrong figures, will the right answers come out?’ ... I am not able rightly to apprehend the kind of confusion of ideas that could provoke such a question.”

Charles Babbage

Chapter 5

Structuring the Code

Motivation

- We could throw all the code into one big file
- For instance, by prompting
 - Write a program that reads data from CSV-files, cleans it, computes statistics, detects outliers, ...
 - Implement the above specification
- An LLM would happily create a single program containing hundreds of lines of code
- Program may work correctly but why is this a bad idea?

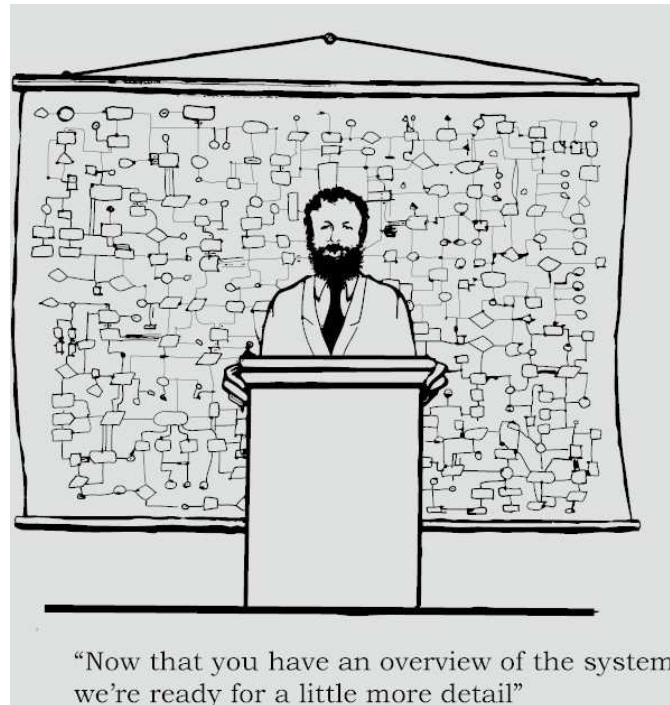
Motivation (2)



- Code like this is hard to follow and maintain
- This is the case not just for humans
- An LLM will also have problems
 - The size of the context window is limited
 - If the program becomes too large, it will not fit
 - So, an LLM may be missing important information

Decomposition

- Specification says what and how things are done
- Decomposition breaks functionality into pieces and sets up boundaries between them
 - Determines what is done where
 - Defines how pieces interact



Architecture vs. Design

- Software developers use different terminology when it comes to structuring code
 - Architecture: high-level view of a system's structure
 - Design: more concerned with individual components
- There is no strict boundary between the two
- We're not going into details here
 - Important for you: you need to structure the code

Modules

- In Python, modules structure your code
- A module is simply a file containing Python code
- E.g., we could introduce the following structure:

```
project/  
├── data/  
│   ├── loader.py  
│   └── cleaning.py  
├── analysis/  
│   ├── statistics.py  
│   └── outliers.py  
├── reporting/  
│   └── results.py  
├── visualization/  
│   └── plot.py  
└── main.py
```

Modules (2)

- It's not about how many files there should be
- In software development, modules are about
 - responsibilities
 - interfaces
 - dependencies
- In agentic coding, they also set boundaries

Responsibilities

- Ideally, a module should perform one specific task
- Questions to ask yourself are:
 - Does every module have a clear purpose?
 - Are any modules doing too many unrelated things?
 - Are there unnecessary modules?

Interfaces

- The interface defines how the outside world interacts with a module
- Should not reveal internal implementation details
- States which services a module provides
- Questions to ask yourself are:
 - What does each module expose?
 - Are the interfaces simple?
 - How much do modules need to know about each other?

Dependencies

- A module A depends on module B if it relies on the functionality of B to work correctly
- There is also indirect, transitive dependency
- High dependency makes code hard to change or test independently
 - Which modules depend on which other modules?
 - Are there circular dependencies?
 - Could a change in one modules unnecessarily affect many others?

Agent Boundaries

- Code structure defines the working environment
- An agent is able to work locally:
 - A module's implementation can be changed without affecting the environment outside
 - A good interface tells an agent what it may rely on without looking at the inner workings

Agent Boundaries (2)

- This has an impact on
 - where an agent searches for making changes
 - what the agent changes
 - how much unrelated code it can affect
 - how easily changes can be reviewed
- You can explicitly tell an agent to make changes to only one module (or few modules)
- Ultimately, this lowers the amount of information needed in the context window

Agent Boundaries (3)

- Questions to ask yourself are:
 - If an agent is told to modify functionality, which file should it look at (and is this obvious)?
 - Can it make the change without modifying a number of other modules?
 - May an agent accidentally cross boundaries?

Summary

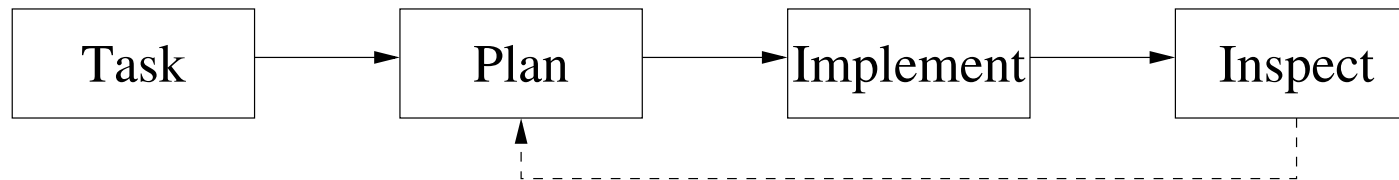
- Don't let the agent turn a specification directly into a pile of code.
- First, decide how to divide the problem into components and decide where the boundaries are
- Then let the agent implement within these boundaries
- You can let the agent propose a code structure, but you should not delegate all decisions to it
 - For larger projects, agents have a tendency to create too many modules

Chapter 6

Implementation

From Design to Implementation

- So far, we have problem → specification → structuring
- However, the structure is not an implementation plan
 - Don't just tell the agent: now build all this
- Manage the agent via an implementation loop



- Again, very similar to agentic loop

Subtasks

- Divide implementation into (independent) subtasks
- Structure should help in this; if it doesn't, you may want to reconsider the structure
- Ideally, we have the following:
 - The goal is clear
 - The relevant requirements are known
 - The scope is bounded
 - There is a clear completion criterion that can be inspected and tested
- This is closely related to the specification

Example

- Implementing the data loader is a good subtask
 - Goal: load CSV file from disk into memory
 - Requirements: file name and format of input file, we also need to know format in main memory
 - Scope: all we do is load data from disk to memory
 - Correctness: all data has been correctly loaded

Planning

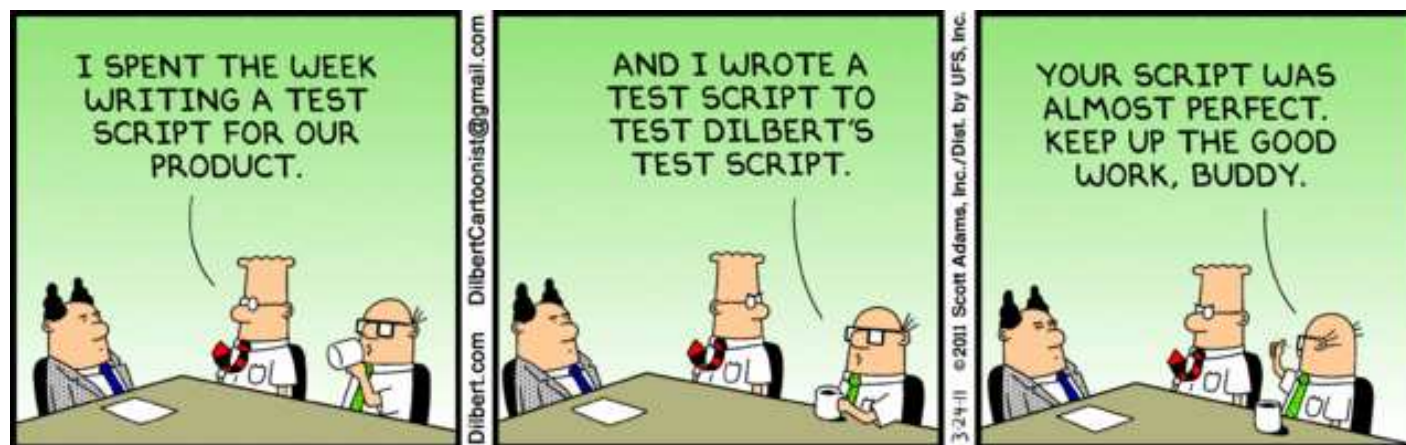
- LLMs seem to be eager to generate code
- Tell it not to implement the code yet, but to do some planning first
 - Many agents have a read-only plan mode (`/plan`)
- In the planning phase, ask the agent to
 - inspect the relevant code (if there's already code)
 - explain the proposed approach
 - identify files that are changed
 - point out any ambiguities
 - generate tests to check correctness

Implementation

- Once you're happy with the plan, then you can tell the agent to implement
- This step does not need any interaction
- However, you can still follow the output as the agent generates the code
- You may catch something that looks strange

Inspecting Changes

- Don't trust the explanations of the agent
- You should run some tests on the code
 - Tests should be written/generated before the code
 - As this is an important topic, we provide more details in the following chapter
- Depending on the outcome, you do different things



Failing Tests

- You need to diagnose the failure
 - You can ask the agent to help you
 - Provide details of the test outcome
 - Ask it determine why it happened
 - You can also look at the code changes
 - Request explanations from the agent
- Tell the agent to fix the code
 - It should not change the tests

Succeeding Tests

- Even if all tests succeed, it may still be a good idea to look at the code yourself
- First, you have to find the changes
 - Git has the command `diff` to show all changes
 - You can also ask the agent to point them out
- Then review them yourself
- Check if important corner cases are missing

Reviewing Code

- For example, `diff` returns the following snippet

```
calculate_average(values):  
    return sum(values) / len(values)
```

- Looks reasonable

Reviewing Code

- For example, `diff` returns the following snippet

```
calculate_average(values):  
    return sum(values) / len(values)
```

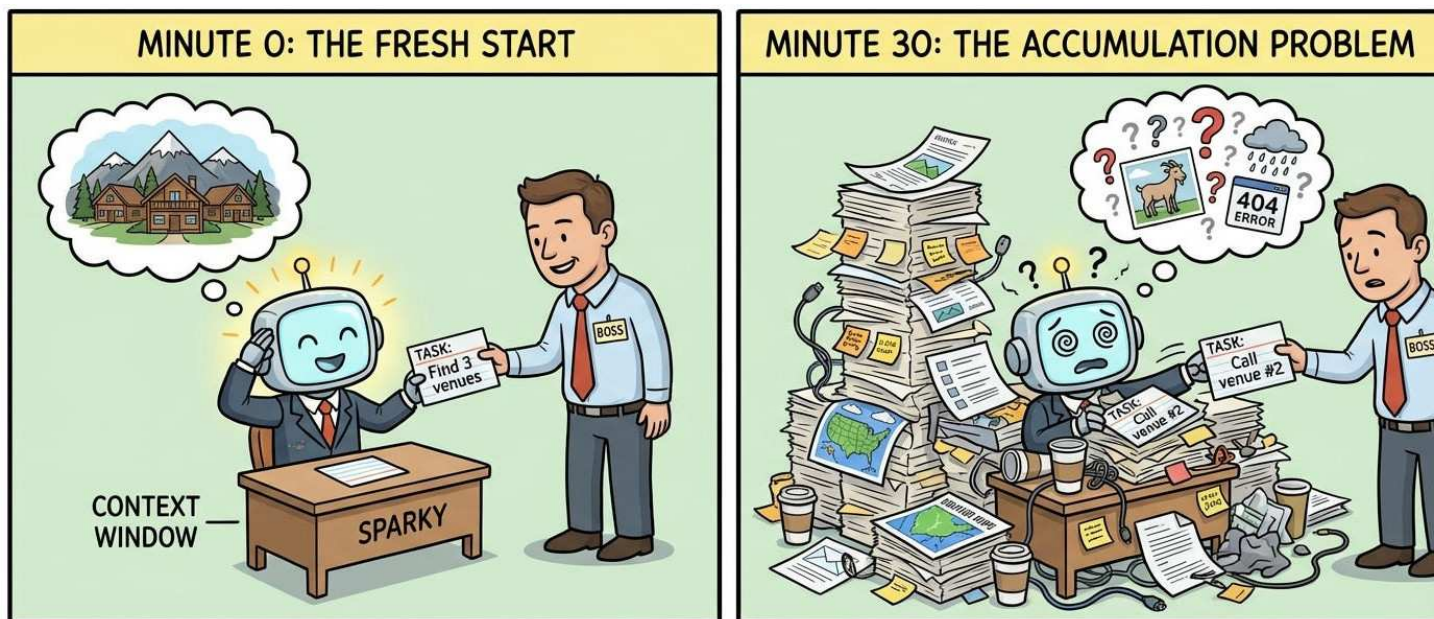
- Looks reasonable
- What happens if `values` is empty?

Context Management

- We mentioned that an agent's context memory is limited
 - Every irrelevant piece of information creates noise
- Implementing code, an agent has to consider a lot:
 - Specification
 - Source code and its structure
 - Tests
 - Documentation
 - Previous conversations
 - Tool output
 - Error messages
- Not all of this should be included in every interaction

Context Management (2)

- Provide the context needed for the current decisions
 - Current subtask
 - Relevant parts of the specification
 - Relevant code files
 - Relevant acceptance criteria



When the Agent Gets Stuck

- You might go through the implementation loop several times unsuccessfully
- If this happens, it's time to revisit certain aspects
 - Was the subtask too large?
 - Did you review the code thoroughly enough?
 - Are the tests wrong?
 - Did the agent misunderstand the structure?
 - Is the specification too ambiguous?

Code Repositories

- Code repositories, such as Git, store development snapshots you can go back to
- For the moment, we recommend not to give an agent direct access to the repository
 - Keep control by manually committing code
- Commit working states that are meaningful, i.e., you have successfully completed an intermediate step
- If an agent messes up, you can go back to that state

Summary

- Generating code is only a part of the workflow
- Also involves determining subtasks, planning, verification, reviewing, and debugging
- Usually, you have to go through several iterations
- An agent isn't replacing the development process
- It is accelerating it

Chapter 7

Test-Driven Development

Testing

“Beware of bugs in the above code; I have only proved it correct, not tested it.”

Donald Knuth

- Testing is making sure our code does what we intend
- For the purposes of this class, we focus on *unit testing* (https://en.wikipedia.org/wiki/Unit_testing)
 - Basic and most important building block of most testing strategies

Example Test

- A unit test verifies the correct operation of a small piece of code, often a single function.

```
import pytest # Import Python's unit testing framework
from addressverifier import AddressVerifier
# Import the module and class that we want to test

def testValidSwissAddress():
    # Call verifier with an address that should pass verification
    result = AddressVerifier().verify(
        street="Rämistrasse 71", city="Zürich", canton="ZH",
        postalCode="8006", country="CH"
    )
    assert result.isValid
    # "assert" makes the test fail when the statement is false
    assert result.errors == []
```

Another Test

- Code that accepts all addresses as valid would pass the test on the previous slide
- We can add a test for an address that should fail:

```
import pytest # Import Python's unit testing framework
from addressverifier import AddressVerifier
# Import the module and class that we want to test

def testInvalidSwissAddress():
    # Call verifier with an address that should fail verification.
    result = AddressVerifier().verify(
        street="Rämistrasse 71", city="Zürich", canton="HH",
        postalCode="80060", country="CH"
    )
    assert not result.isValid
    # "assert" makes the test fail when the statement is false
    assert len(result.errors) == 2 # Expect to find two errors
    assert "NoSuchCanton" in result.errors # non-existent canton
    assert "BadPostCode" in result.errors # Invalid postcode
```

Test-driven development (TDD)

- Writing the tests first can be useful because it makes you think about how your interface will be called:
 - what data will a caller provide?
 - which return values will make sense to them?
 - which error codes will be useful?
- Encourages you to test your *interface* rather than your *implementation*.

TDD Process

- Work on one feature at a time
- A feature can be
 - an interface method
 - or even one behavior of an interface method
 - For example, "Postcodes can be specified as XXXX or CH-XXXX"
- Write tests for that one feature.
- Run the tests and verify that they fail.
- Fix the code until all the tests pass.

TDD Process (2)

- Reviewing tests is easier than reviewing code!
- You can ask the coding agent to write the tests first
- You review the tests, and only then let the agent modify the code

Test coverage

- How much testing is enough?
- *Test coverage* measures how many of your project's lines of code get executed by tests
- Gives you a rough idea how much of your functionality is getting tested.
- Between 70% and 80% line coverage is a good goal
 - Higher coverage has diminishing returns
- Ask the agent to check and maintain test coverage
- This is included in our sample *AGENTS.md* file

What Do Good Tests Look Like?

- Good tests give you confidence:
 - Run automatically
 - Complete quickly
 - Let you catch mistakes early
- Good tests
 - focus on a single feature
 - test the *interface*, not the *implementation*
 - run quickly and without external dependencies, i.e.,
 - no access to the internet or production data stores

What Do Bad Tests Look Like?

- Bad tests fail a lot
 - You end up fixing the tests rather than the code
- This mainly happens for two reasons:
 - *brittle* tests are tied to the *implementation* rather than the *interface*
 - Need to be changed even for small code changes
 - *flaky* tests depend on behavior or state outside of the scope of the system under test
 - For example, an external data store
 - Tend to be slow and to fail randomly

External Dependencies

- What happens if you have external dependencies?
 - For example, you test code that accesses a database
- Mocks are your friend:
`https://pytest-mock.readthedocs.io/en/latest/`

Tests Written By Coding Agents

- Absent careful prompting or relevant *skills*,
 - LLMs often write tests that are both brittle and flaky
- Not a big problem for small, short projects
- Becomes annoying when using and extending the code
- Takeaway:
 - Put good testing practices into the context
 - Review the tests to make sure they stay useful

Summary: TDD

- Good tests are a recipe for useful software
- They ensure that
 - new code satisfies new requirements
 - modifications to existing code don't break existing features
- Tests are easiest to write for well-structured code with clear interfaces
 - Difficulties in creating small independent tests can point to a problem in the interface design

Chapter 8

Debugging and Refactoring

Debugging

- Eventually, your code will contain a bug
 - That means, it doesn't produce the expected output
- Proper reaction is not just telling the agent: fix this
- It should be resolved in the following way:
 - Reproduce the failure reliably (what)
 - Then, investigate the cause (why)
 - Finally, fix the problem

Example

- Let's go back to the example for calculating averages

```
calculate_average(values):  
    return sum(values) / len(values)
```

- Your program just crashed, outputting

```
File "...", line x, in calculate_average  
ZeroDivisionError: division by zero
```

- Give this error message to the agent and ask it
 - to investigate what is going on and
 - to hypothesize why it is happening

Investigate

- Have a look at the agent's answer
 - Does it explain what is happening?
- Clearly, a division by zero triggers the error
 - But why is `values` empty?
- Can this error be reproduced with other inputs?
 - This further evaluation can be supported by the agent again

Example

- The agent may come back with the following answer
 - The input file did not pass the validation
 - This resulted in `values` not being properly initialized
- This can be tested by using corrupt input files that cannot be validated

Preparing Modification

- Once you're happy with the diagnosis, you can ask the agent to prepare the modification
- Review the modification
- If you're satisfied, ask the agent to modify the code
- Also test the code!

Testing

- Ideally, the test for the modification is generated before the modification itself
 - Clearly, it will fail without the fix
 - Goal is to make it pass
- Also run the other tests to check that nothing else was broken
 - This is called *regression testing*

Traditional vs Agentic Debugging

All done by developer:

- Investigate
- Hypothesize
- Run experiment
- Evaluate
- Modify code
- Test

- Developer
 - Agent
- Describe evidence
 - Investigate
 - Hypothesize
- Evaluate
 - Run experiment
 - Propose modification
- Review
 - Modify
 - Test

Refactoring

- There may be another reason to modify code
 - The code gives correct results
 - But it is messy, ugly, and hard to understand
- What do we mean by this?
 - One large function doing all the work
 - Duplicated logic
 - Unclear and confusing naming
 - Deeply nested conditionals
 - Unrelated responsibilities in one module
 - Weak abstraction boundaries

Refactoring (2)

- *Refactoring* is about giving code a better structure **without** changing its behavior
 - Debugging changes behavior and structure
- Testing is also very important when refactoring
 - Don't refactor code that isn't covered by tests
- Typical workflow of refactoring
 - Understand behavior
 - Run existing tests
 - Identify missing tests and add tests if necessary
 - Refactor
 - Run tests again
 - Evaluate

Agentic Refactoring

- Again, it's not enough to just say: refactor this
- Be specific in your instructions
- For example, you want to refactor the module `Parser`, then you may want to give these instructions:
 - Refactor `Parser` to separate parsing from validation
 - Preserve the API and the observable behavior
 - Do not change the file format or error semantics
 - Run the tests after each significant change
- Looks similar to specifications and constraints

Agentic Refactoring (2)

- After agent finishes refactoring, verify the results
 - Run all available tests
 - Check the differences in the code
 - Manually check behavior of critical parts
- The effort you have to make depends on modifications
 - Less for a few variable name changes
 - More for a rewrite of a core component

Summary

- Debugging/refactoring follows similar workflow as agentic implementation
 - Do not let the agent do everything on its own
 - Embed the work into a feedback loop



